

---

# Modül 09 — Hazelcast ile Dağıtık Veri

Ödevler

Spring Boot Edu



2026-09-23

## İçindekiler

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>Nasıl Çalışılır</b>                                 | <b>2</b> |
| <b>Ödev 1 — Near Cache’li Bir Client (Kolay)</b>       | <b>2</b> |
| <b>Ödev 2 — Bütün Siparişi Ayırmak (Zor)</b>           | <b>3</b> |
| <b>Ödev 3 — Redis mi, Hazelcast mi? (Yazılı Rapor)</b> | <b>3</b> |
| <b>Ek Meydan Okuma (İsteğe Bağlı)</b>                  | <b>4</b> |

## Nasıl Çalışılır

1. Başlangıç kodu `modules/09-hazelcast/exercise/` altındadır. TODO yorumlarını bulun.
2. Testler kendi embedded Hazelcast üyelerini başlatır. **Docker gerekmez.**
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/09-hazelcast/exercise -am test
```

4. Bir ödevin tüm testleri yeşil olduğunda ödev tamamdır. Ödev 3 yazılı bir rapordur ve testi yoktur.
5. Takılırsanız önce ipuçlarını, sonra `modules/09-hazelcast/solution/` altındaki çözümü okuyun.

### ★ İpucu

TestMembers (test kaynaklarında), kendi cluster adıyla bir üye başlatır. Böylece test sınıfları asla birbirine katılmaz.

## Ödev 1 — Near Cache’li Bir Client (Kolay)

**Hedef:** Kitap verisinin tekrar okunmasını client içinde yerel yapmak ve farkı ölçmek.

Test bir üye başlatır ve client’ınızı ona bağlar. ReadTimer hazır verilmiştir: Aynı anahtarı near cache’li books map’inden ve near cache’i olmayan orders map’inden birçok kez okur.

**Yapılacaklar** (`exercise1.NearCacheClient.create`):

- TODO 1a — **Yalnızca** books map’i için, serileştirilmemiş nesneleri tutan bir near cache.
- TODO 1b — Yerel kopyaların süresi 60 saniyede dolsun ve kayıt cluster’da değişince geçersiz kılınsın.
- TODO 1c — Near cache’i client yapılandırmasına ekleyin.

### İpuçları:

- Ders bölüm 3.6: `new NearCacheConfig("books"), setInMemoryFormat(InMemoryFormat.OBJECT), setTimeToLiveSeconds(...), setInvalidateOnChange(true).`
- `config.addNearCacheConfig(...).`
- Test, ReadTimer sonucunu yazdırır. İki sayıyı karşılaştırın: Near cache sizin bilgisayarınızda kaç kat daha hızlı?

**Kabul kriterleri:** `Exercise1Test` içindeki 4 testin hepsi geçer.

**Tahmini süre:** 20 dakika

## Ödev 2 — Bütün Siparişi Ayırmak (Zor)

**Hedef:** Birden çok kitap içeren bir sipariş, aynı anda birçok sipariş çalışırken ya tamamen ayrılsın ya da hiç ayrılmasın.

**Yapılacaklar** (exercise2.OrderReservation.reserveAll):

- TODO 2a — Siparişteki her kitabın anahtarını deadlock'ü önleyen bir sırayla kilitleyin.
- TODO 2b — Herhangi bir kitabın stoğu sipariş edilenden azsa hiçbir şeyi değiştirmeyin ve false döndürün.
- TODO 2c — Aksi hâlde her kitabın stoğunu düşürün ve true döndürün.
- TODO 2d — Bir şeyler ters gitse bile aldığınız her kilidi bırakın.

### İpuçları:

- Ders bölüm 3.5: try/finally içinde stock.lock(isbn) / stock.unlock(isbn).
- **Herhangi birini** değiştirmeden önce **tüm** kitapları kontrol edin.
- A siparişi 1. kitabı kilitler ve 2. kitabı bekler. B siparişi 2. kitabı kilitler ve 1. kitabı bekler. İkisi de ilerleyemez. Herkes kitapları aynı sırayla, örneğin ISBN'e göre sıralı kilitlerse bu olamaz.
- finally tam olarak bunları açabilsin diye hangi anahtarları kilitlediğinizi hatırlayın.

**Kabul kriterleri:** Exercise2Test içindeki 4 testin hepsi geçer. Kodunuz deadlock'a girebiliyorsa ordersThatNameTheSameBooksInOppositeOrderDoNotDeadlock 20 saniye sonra başarısız olur.

**Tahmini süre:** 45 dakika

## Ödev 3 — Redis mi, Hazelcast mi? (Yazılı Rapor)

**Hedef:** Bir gereksinim için doğru aracı seçmek ve seçimi gerekçelendirmek.

1-2 sayfalık bir rapor yazın (kendi notlarınızda report.md; repoya eklenmez). Kitapçı için Redis'i (modül 08) ve Hazelcast'i (bu modül) karşılaştırın:

1. 5 uygulama örneği için **ürün detay önbellegi**: saniyede 1.000 kez okunur, günde birkaç kez değişir.
2. Kampanya sırasında **stok ayırma**: aynı kitaplar için çok sayıda eşzamanlı alıcı.
3. Giriş yapmış kullanıcıların **oturumları**.
4. Tüm örneklere **fiyat değişikliği bildirimleri**.

Her durum için şunları yanıtlayın:

- Hangi aracı, hangi topolojide kullanırdınız (Redis sunucusu, embedded Hazelcast, Hazelcast client-server)?
- Veri eşzamanlı erişimde nasıl doğru kalır (atomik komutlar, kilitler, entry processor'lar)?

- Bir uygulama örneği veya veri deposu yeniden başladığında ne olur?
- Çalıştırmanın maliyeti nedir (ek sunucular, her uygulamada bellek, lisanslar)?

**Değerlendirme rehberi:** İyi bir rapor tek bir kazanan ilan etmez. Her aracın daha uygun olduğu en az bir durumu gösterir ve CP Subsystem'in lisans konusuna değinir.

**Tahmini süre:** 60 dakika

## Ek Meydan Okuma (İsteğe Bağlı)

Ödev 2'yi hiç kilit kullanmadan çözün: Tek bir kitabı ayıran bir `EntryProcessor` yazın ve bunun **birden çok** kitaplı bir sipariş için neden yetmediğini düşünün. `IMap.executeOnKeys` ve Hazelcast transaction API'sine (`TransactionContext`) bakın ve bunları kilit tabanlı çözümünüzle karşılaştırın.